KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous) · Coimbatore – 641 407

Department of Computer Science and Engineering

WEB TECHNOLOGY – ASSIGNMENT II

Course Code & Title: U21CS501 – Web Technology

Maximum Marks: 40

Project Title

Student Admission Portal with Profile & Document Management

HTML · CSS · JavaScript · PHP · MySQL · Java Servlet · JSP · JDBC

Submitted by

Mithuna Kamalanathan

Roll Number	24CS117
Course	B.E Computer Science and Engineering
Year / Semester / Section	III / V / A

Academic Year	2026 – 2027
Issue Date	07.10.2026
Submission Date	20.10.2026

Contents

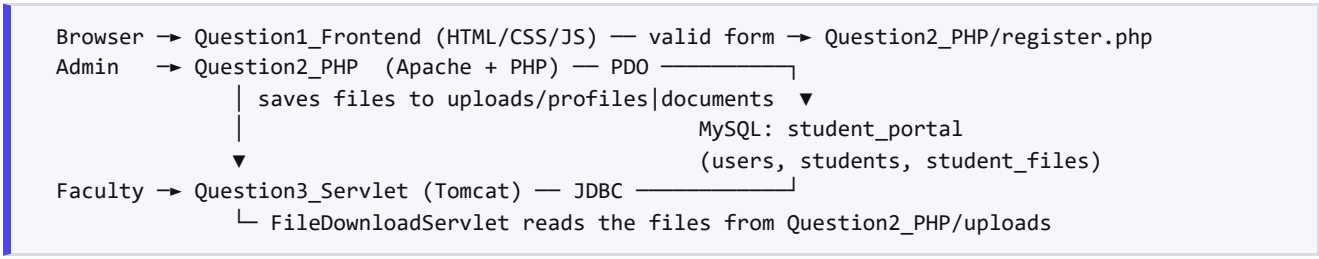
1. Project Overview and Architecture
2. Question 1 – Student Registration Form (HTML, CSS, JavaScript) – 8 Marks
3. Question 2 – PHP & MySQL Admin Panel (CRUD and File Uploads) – 16 Marks
4. Question 3 – Servlet, JSP & JDBC Student Profile Portal – 16 Marks
5. Database Schema
6. Security Measures
7. Testing Summary
8. Conclusion
9. GitHub Repository and Submission Files

1. Project Overview and Architecture

All three questions are implemented as **one student admission portal** made of three separate modules that share a single MySQL database named `student_portal`:

Module (folder)	Question	Technologies	Purpose
`Question1_Frontend`	Q1	HTML5, CSS3, JavaScript	Registration form with client-side validation
`Question2_PHP`	Q2	PHP 8 (PDO), MySQL	Admin panel: CRUD, file uploads, metadata
`Question3_Servlet`	Q3	Java Servlet, JSP + JSTL, JDBC, Tomcat	Session-protected portal that lists profiles and streams files
`database`	All	MySQL / MariaDB	`schema.sql` – tables, constraints, demo data

Flow: a student fills in the registration form → JavaScript validates it → PHP saves it. The administrator logs in to the PHP panel to add, edit, view and delete students and to upload photos and documents. Each file is stored on disk under a random name and its metadata is stored in MySQL. Faculty log in to the Servlet portal, which reads the *same* database through JDBC and streams the *same* files through a Servlet.



Tools: XAMPP 8.2.12 (Apache, PHP 8.2, MariaDB 10.4), Tomcat 8.5.96, JDK 24, Connector/J 8.4.0, JSTL 1.2.5, jBCrypt 0.4.

2. QUESTION 1 – Student Registration Form (8 Marks)

Aim

To design a student admission registration form using HTML, style it with CSS (layout, colours, responsive design, styled inputs, buttons and error messages), and validate it on the client side using JavaScript (required fields, e-mail format and password strength).

Implementation

- `register.html` – the form contains Full Name, Roll Number, Email, Mobile (optional), Course (drop-down), Password, Confirm Password, Profile Photo upload, ID-Proof upload (optional) and a declaration checkbox, with *Submit* and *Reset* buttons. It uses `enctype="multipart/form-data"` so files can be sent, and the `novalidate` attribute so that all validation is done by my own JavaScript.
- `css/style.css` – colours are defined once as CSS variables; CSS Grid gives a two-column page and a two-column form; inputs have rounded borders and a focus ring; buttons have hover states; invalid fields turn red with a message underneath and valid fields turn green. Media queries switch to a single column on tablets and phones.
- `js/validation.js` – one validator function per field. Fields are checked when the user leaves them and again while typing; a password-strength meter and checklist update on every key press. On submit, every field is validated and `event.preventDefault()` stops the form if anything is wrong.

Validation rules

Field	Rule checked in JavaScript
Name	Required, 3–100 characters, letters / spaces / dot / apostrophe only
Roll number	Required, 5–15 letters or digits, no spaces (converted to upper case)
Email	Required, must match `name@domain.tld`
Course	A course must be selected
Password	Required; at least 8 characters with an upper-case letter, a lower-case letter, a number and a special character
Confirm password	Must be identical to the password
Profile photo / ID proof	Allowed extensions (JPG/JPEG/PNG/WEBP for the photo; PDF/JPG/JPEG/PNG for the ID proof), not empty, max 2 MB / 5 MB

Code Explanation

The excerpts below are copied from the project source and shortened; "..." marks omitted lines.

`Question1_Frontend/js/validation.js – password strength rules and password validator`

```
const PASSWORD_RULES = {
    length:  (pw) => pw.length >= 8,
    upper:   (pw) => /[A-Z]/.test(pw),
    lower:   (pw) => /[a-z]/.test(pw),
    number:  (pw) => /[0-9]/.test(pw),
    special: (pw) => /[^A-Za-z0-9]/.test(pw)
};
...
password: function () {
    const password = document.getElementById('password').value;
    if (password === '') { return 'Password is required.'; }
    const missing = [];
    if (!PASSWORD_RULES.length(password))  { missing.push('8+ characters'); }
    if (!PASSWORD_RULES.upper(password))   { missing.push('an upper-case letter'); }
    ...
    if (missing.length > 0) { return 'Weak password – add ' + missing.join(', ') + '.'; }
    return '';
},
```

Question1_Frontend/js/validation.js – blocking an invalid submission

```
form.addEventListener('submit', function (event) {
    const errors = [];
    Object.keys(validators).forEach(function (fieldId) {
        if (!validateField(fieldId)) {              // shows the red message under the field
            errors.push(document.getElementById(fieldId + '-error').textContent);
        }
    });
    if (errors.length > 0) {
        event.preventDefault();                      // STOP – the form is not sent
        /* ...build the error summary with textContent and focus the first wrong field... */
    }
});
```

Every validator returns an error message or an empty string. `validateField()` writes the message with `textContent` (never `innerHTML`, so no HTML can be injected) and toggles the `is-invalid` / `is-valid` CSS classes. The source files contain inline comments explaining each part.

Output

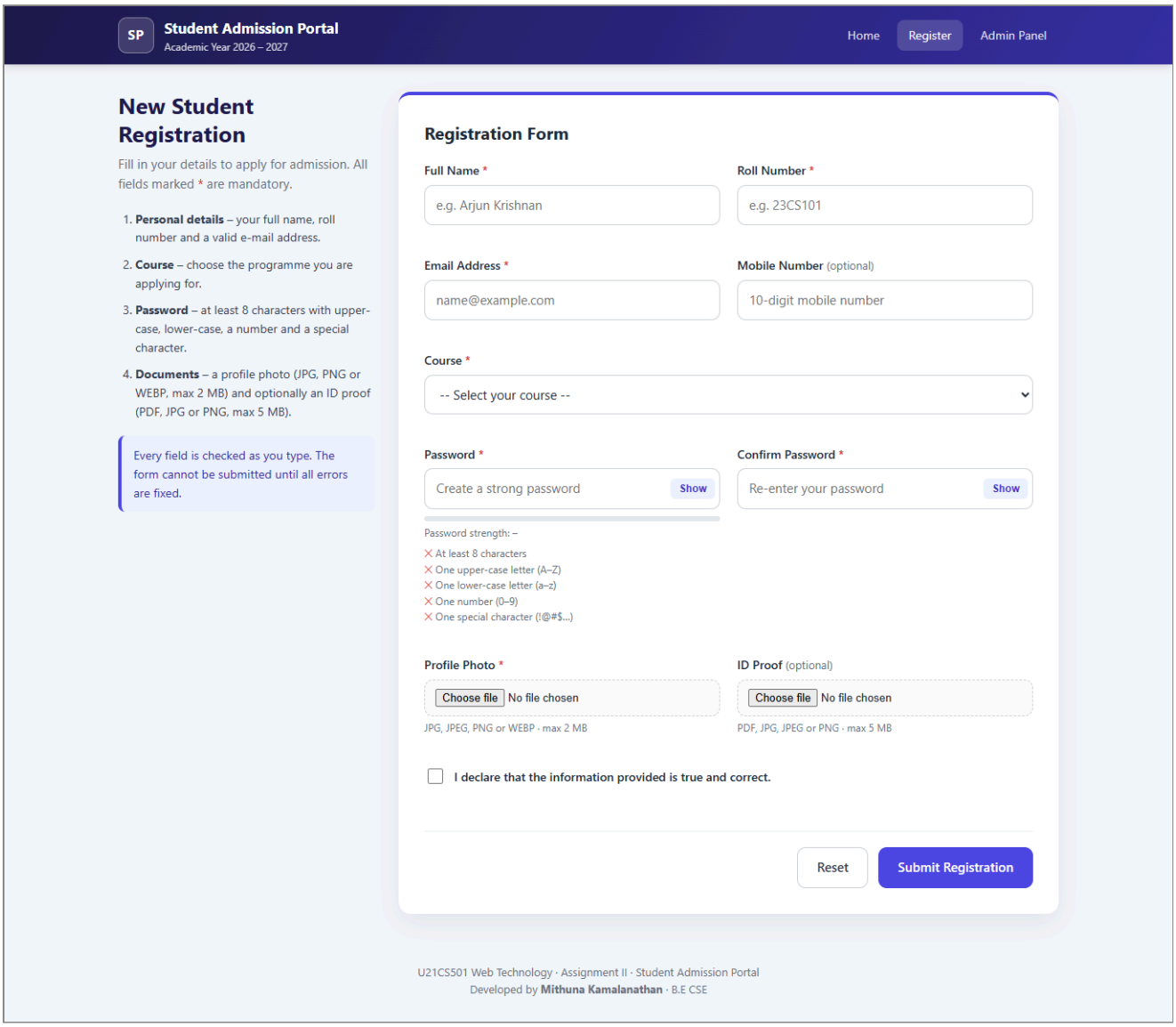


Figure 2.1 – Registration form (HTML + CSS)

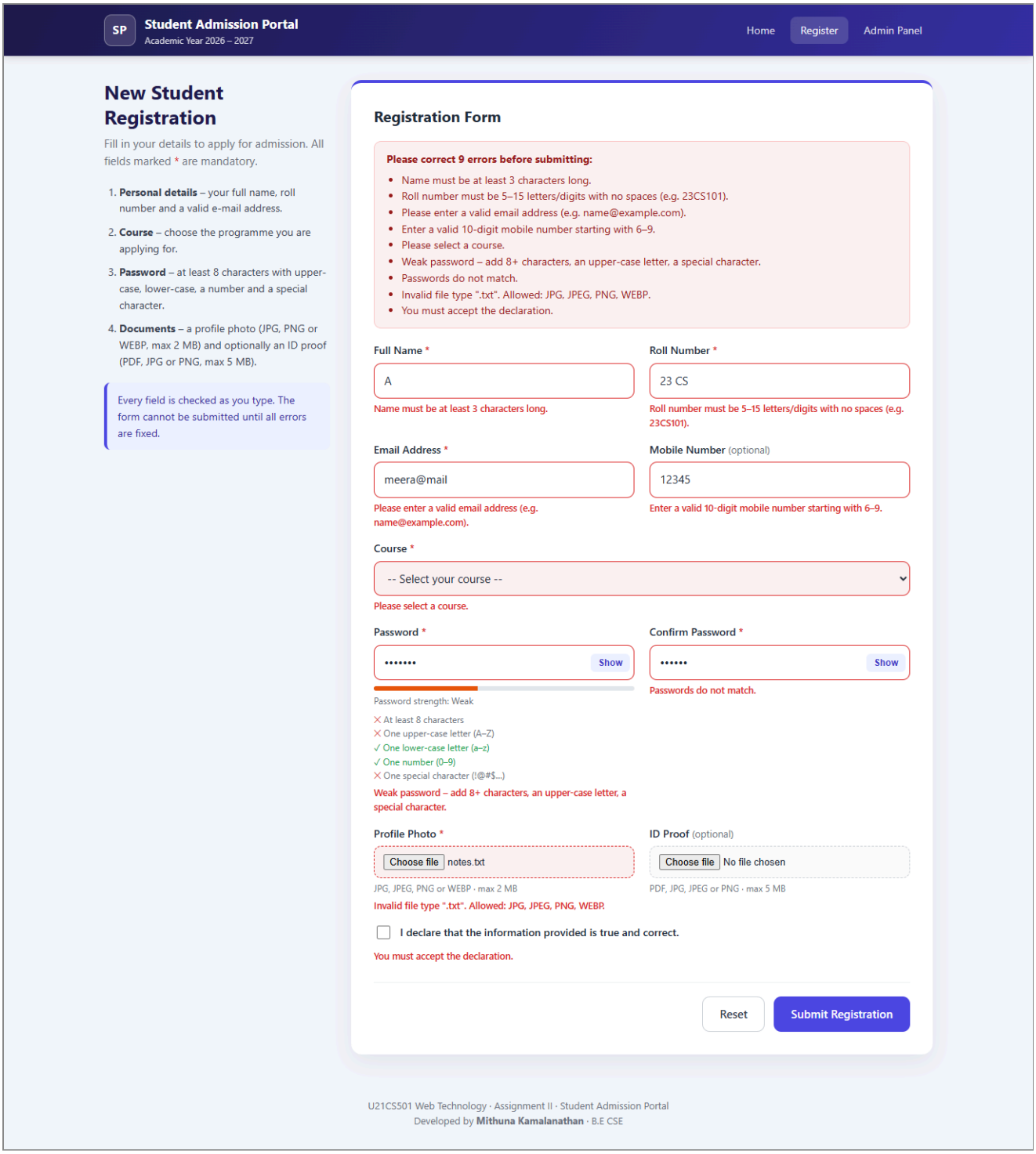


Figure 2.2 – JavaScript validation: error summary, invalid email, weak password, password mismatch, invalid file type; submission blocked

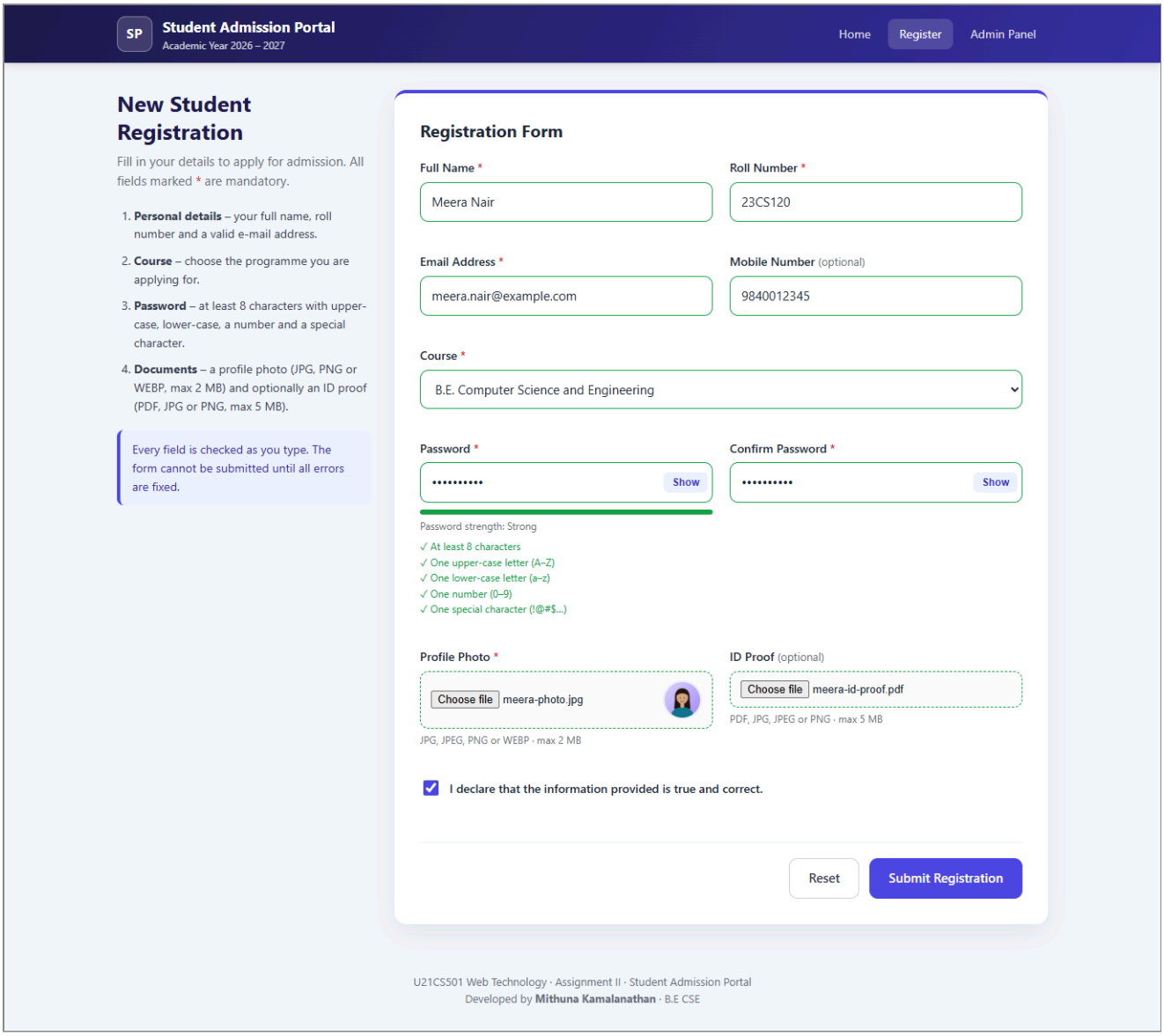


Figure 2.3 – Valid input: all fields green, password strength "Strong", photo preview

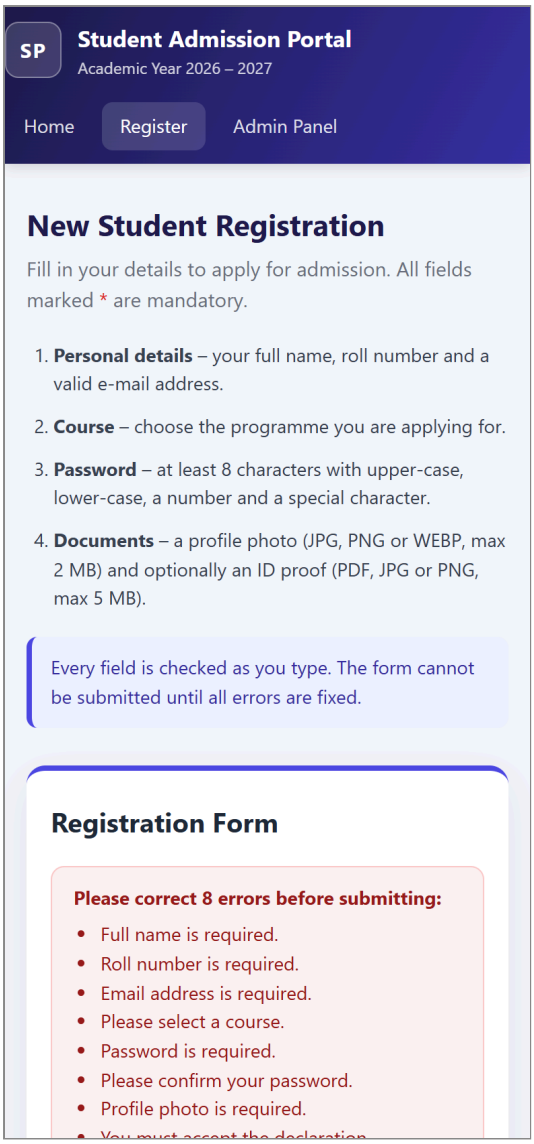


Figure 2.4 – Responsive layout on a mobile screen

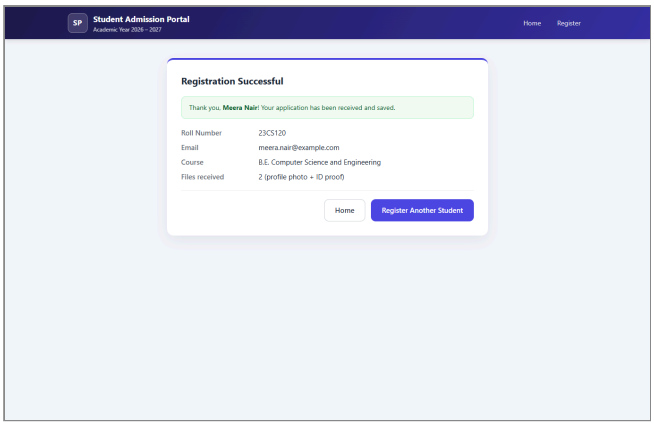


Figure 2.5 – Valid registration saved by PHP

Result

> The student registration form was created with HTML, styled with responsive CSS and validated with JavaScript. Required fields, e-mail format, password strength, password confirmation and file type/size are checked, clear error messages are shown, and the form cannot be submitted until every error is corrected.

3. QUESTION 2 – PHP & MySQL Admin Panel (16 Marks)

Aim

To create PHP pages for the college admin panel that perform CRUD operations on student records in MySQL, handle uploads of profile images and documents, store the file metadata in MySQL linked to the student, display the uploaded files when a student is viewed, and validate file size and type while preventing files from being overwritten.

Implementation

Page	Function
`login.php` / `logout.php`	Admin login with `password_verify()` and PHP sessions; logout destroys the session
`dashboard.php`	**Read** – statistics, search/filter and the list of all students
`add_student.php`	**Create** – new student with optional photo and document
`edit_student.php`	**Update** – loads the record, validates and updates it
`delete_student.php`	**Delete** – confirmation page, then deletes the record and its files
`view_student.php`	**View** – full details, profile photo and all uploaded files
`upload_file.php` / `delete_file.php`	Upload a new photo/document; delete a single file
`view_file.php`	Serves a stored file to the logged-in admin (view or download)
`includes/upload_handler.php`	All upload validation and storage logic
`config/database.php`	MySQL settings and the shared PDO connection

File upload handling and validation

- Upload error codes are checked, and `is_uploaded_file()` confirms the file came from the form.
- **Size:** profile photo ≤ 2 MB, document ≤ 5 MB.
- **Type:** extension whitelist (photo: jpg, jpeg, png, webp; document: pdf, jpg, jpeg, png), dangerous double extensions such as `shell.php.jpg` are rejected, and the **real MIME type** is read from the file content with `finfo`.
- **No overwriting:** every file is saved under a random 32-character name generated with `random_bytes()`, so two files with the same original name never replace each other.
- Files go to `uploads/profiles` or `uploads/documents`; an `.htaccess` file blocks direct access and script execution there.

Code Explanation

The excerpts below are copied from the project source and shortened; "..." marks omitted lines.

Question2_PHP/includes/upload_handler.php – checking size and the real MIME type

```
if ($size > $rules['max_size']) {
    return ['ok' => false, 'error' => sprintf('File is too large (%s). Maximum allowed size is %s.',
            format_bytes($size), format_bytes($rules['max_size']))];
}
...
// 6. Detect the REAL MIME type from the file's bytes (magic numbers)
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = (string) $finfo->file($file['tmp_name']);
if (!in_array($mime, $rules['types'][$extension], true)) {
    return ['ok' => false, 'error' => sprintf('The file content (%s) does not match the .%s
extension.', $mime, $extension)];
}
```

Question2_PHP/includes/upload_handler.php – unique name + metadata stored in MySQL

```
do {   // random, unguessable name – the loop guarantees we NEVER overwrite a file
    $storedName = bin2hex(random_bytes(16)) . '.' . $validated['extension'];
    $destination = $folder . DIRECTORY_SEPARATOR . $storedName;
} while (file_exists($destination));

move_uploaded_file($file['tmp_name'], $destination);

$stmt = db()->prepare(
    'INSERT INTO student_files
         (student_id, file_category, document_label, original_name, stored_name, file_path,
file_type, file_size)
     VALUES (:student_id, :category, :label, :original_name, :stored_name, :file_path, :file_type,
:file_size)');
$stmt->execute([':student_id' => $studentId, ':file_path' => $relativePath, ':file_type' =>
$validated['mime'], ...]);
```

Question2_PHP/edit_student.php – UPDATE with a prepared statement

```
$sql = 'UPDATE students
           SET name = :name, roll_number = :roll, email = :email, course = :course,
               phone = :phone, date_of_birth = :dob, address = :address WHERE id = :id';
$stmt = db()->prepare($sql);
$stmt->execute($params);          // user input is sent separately – no SQL injection
```

Every form is re-validated on the server (`validate_student_data()`), the roll number and e-mail are checked for duplicates, every POST form carries a CSRF token, and every value printed in HTML is escaped with `htmlspecialchars()`. The upload date is filled automatically by MySQL (`DEFAULT CURRENT_TIMESTAMP`).

Output

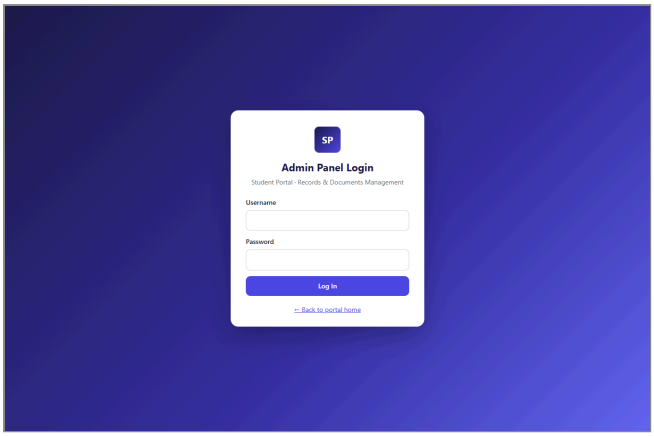


Figure 3.1 – Admin login

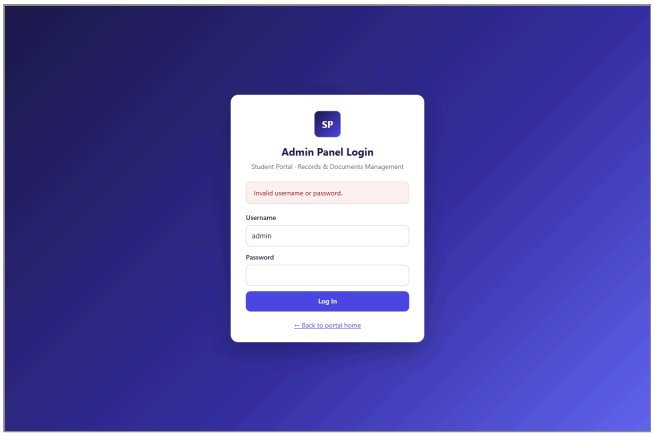


Figure 3.2 – Invalid login

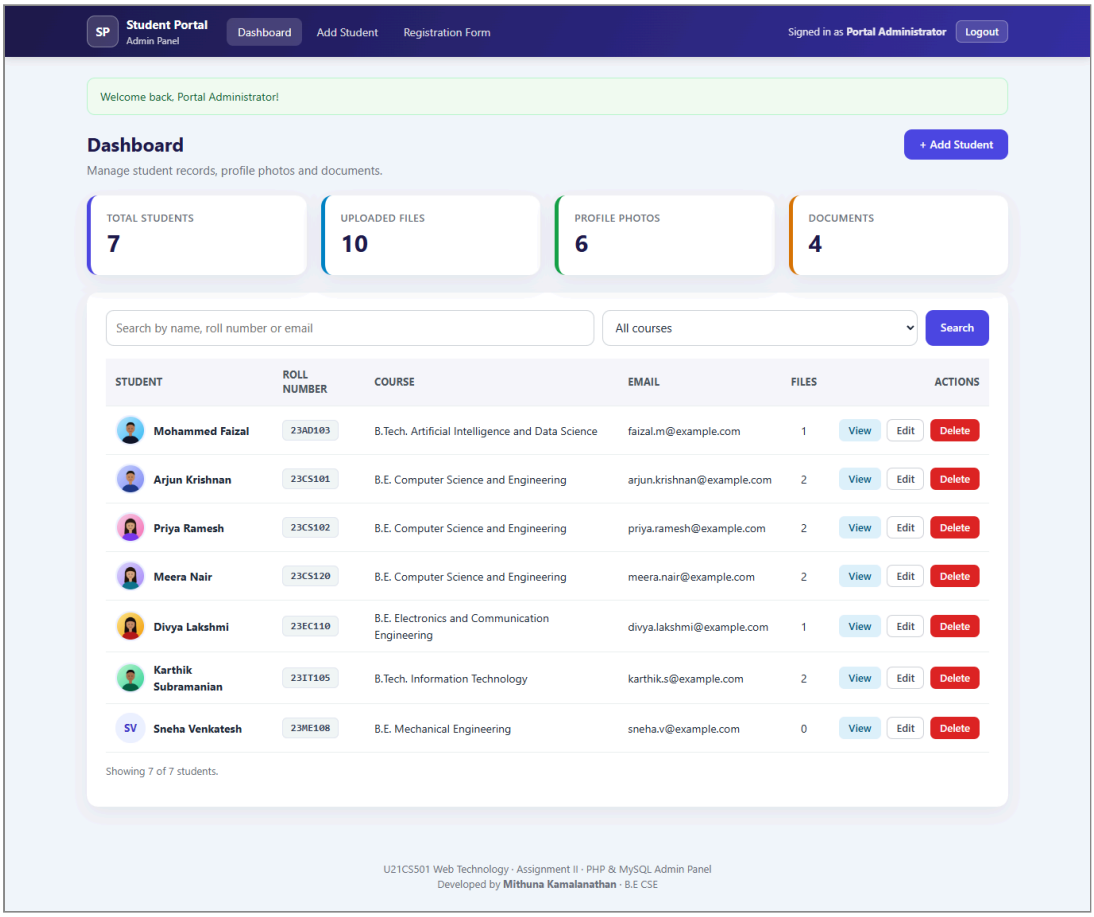


Figure 3.3 – Dashboard (Read): statistics and student list with photo thumbnails

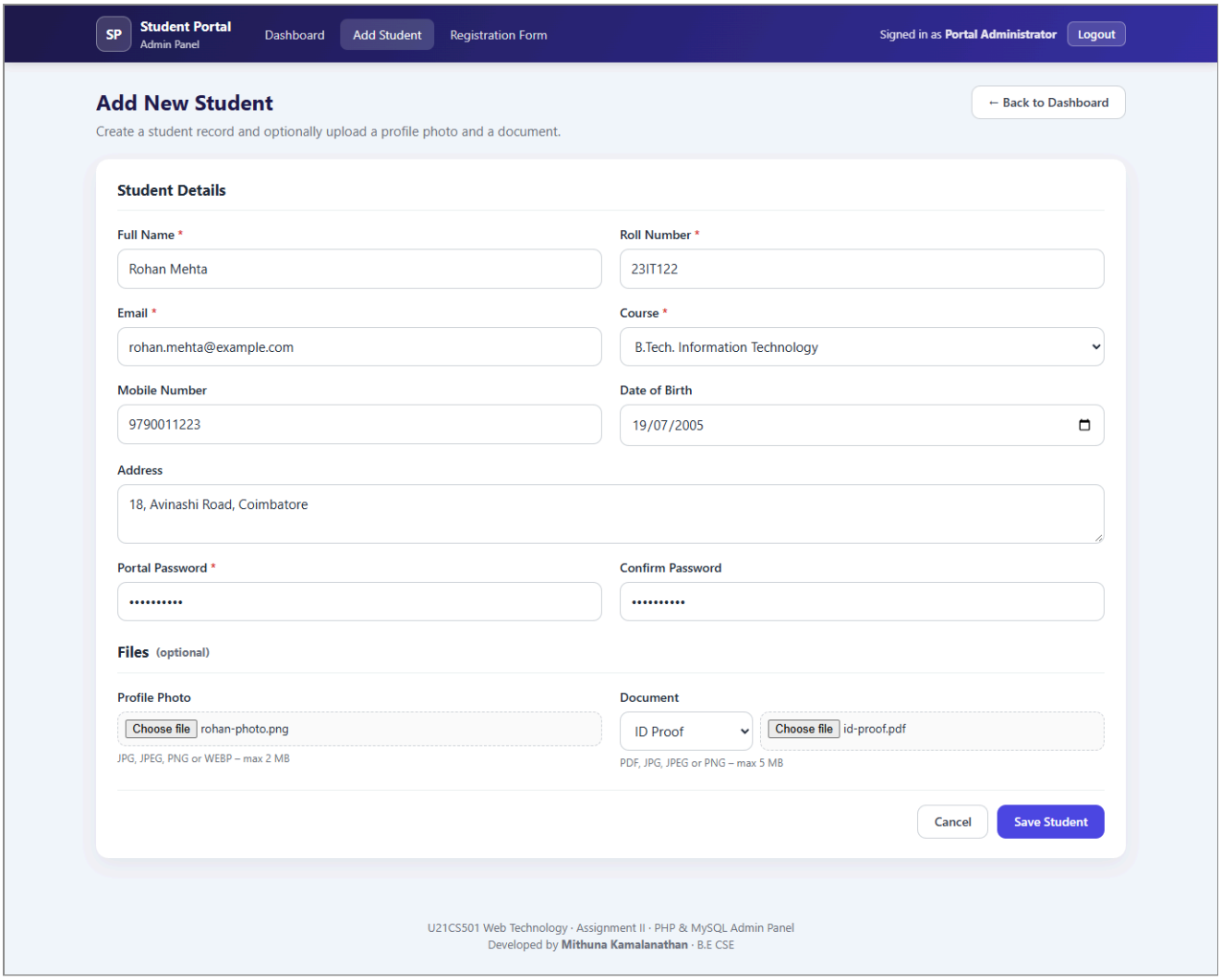


Figure 3.4 – Add Student (Create) with profile photo and document

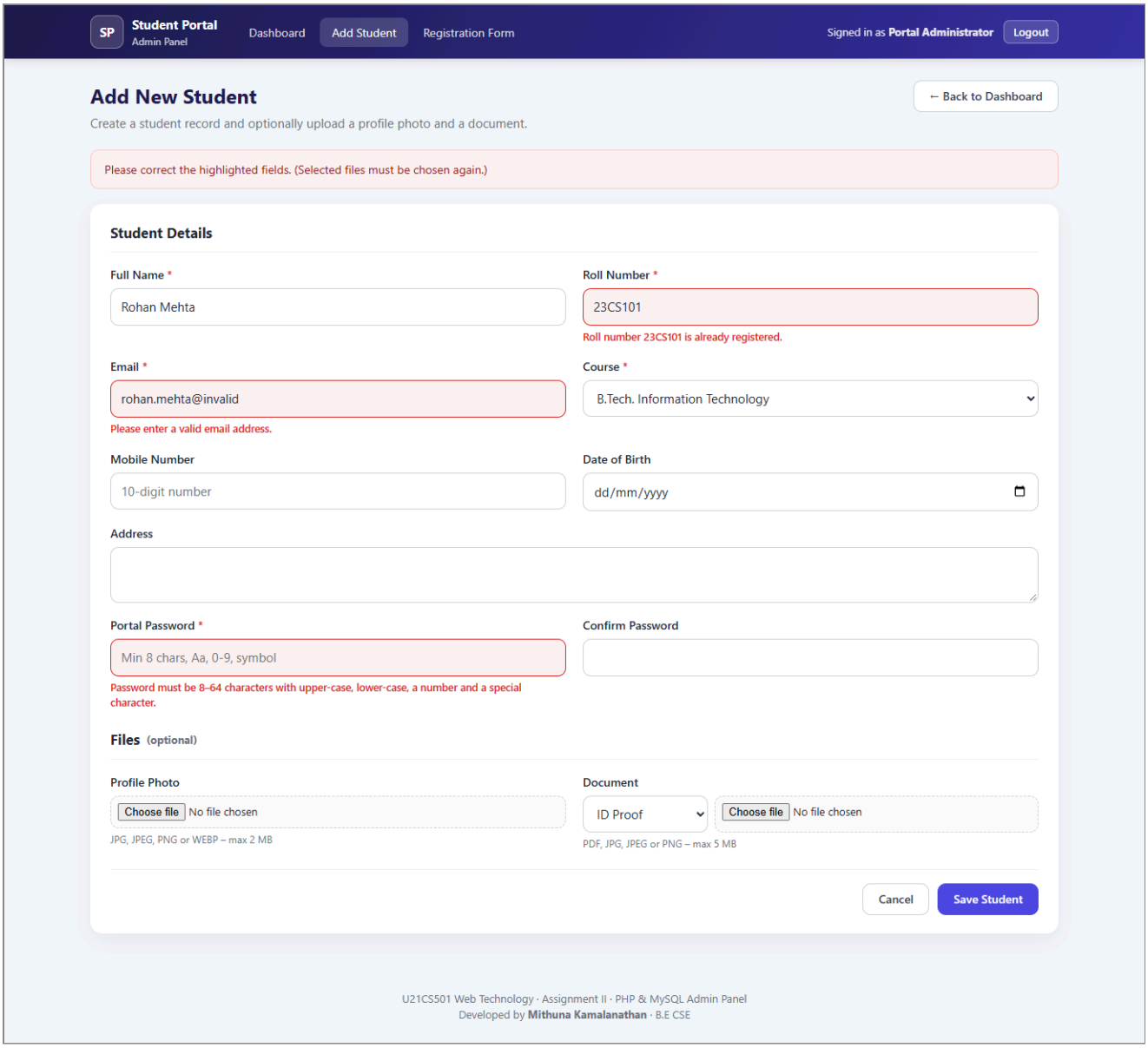


Figure 3.5 – Server-side validation: duplicate roll number, invalid e-mail, weak password

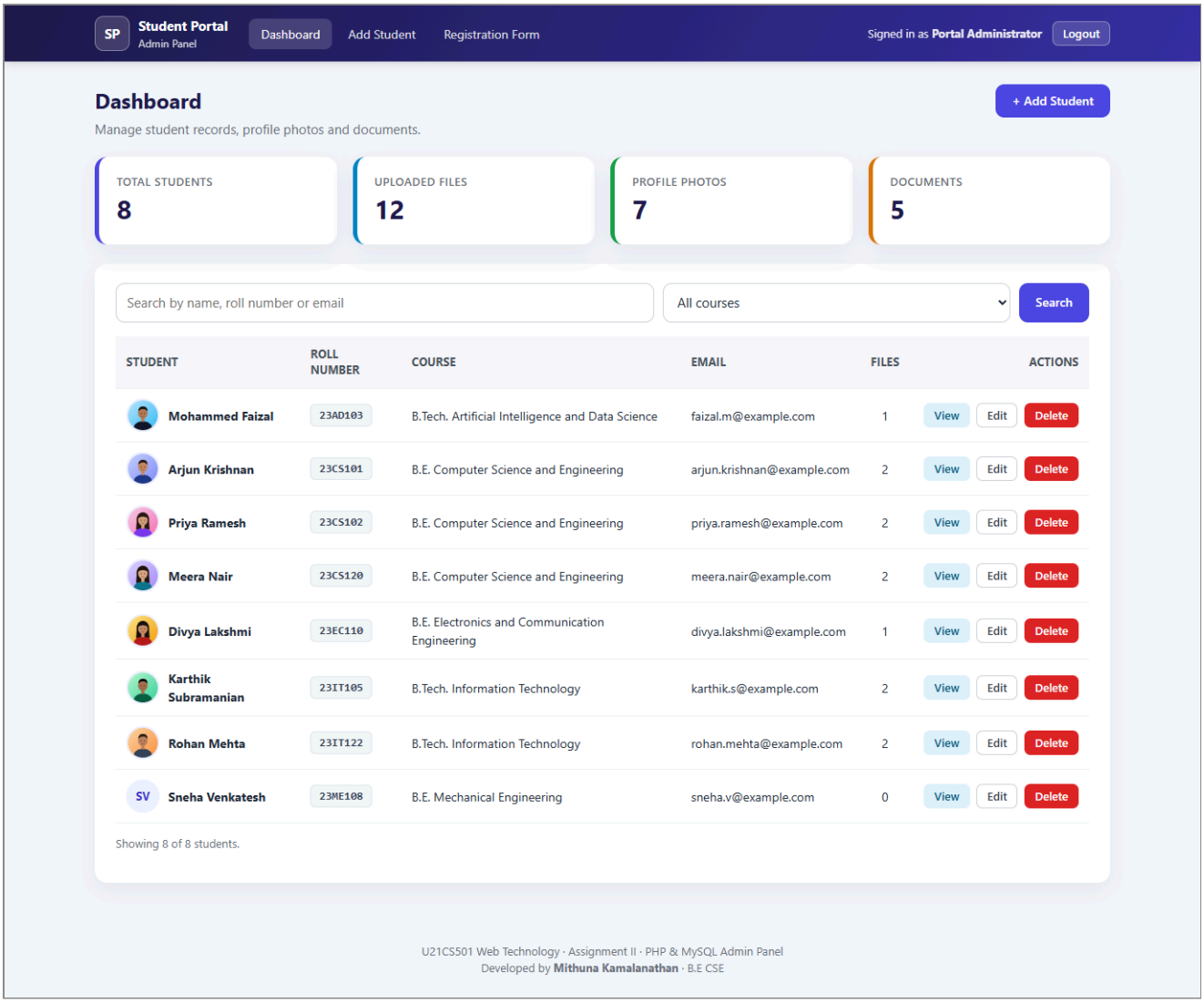


Figure 3.6 – Student list after adding a student

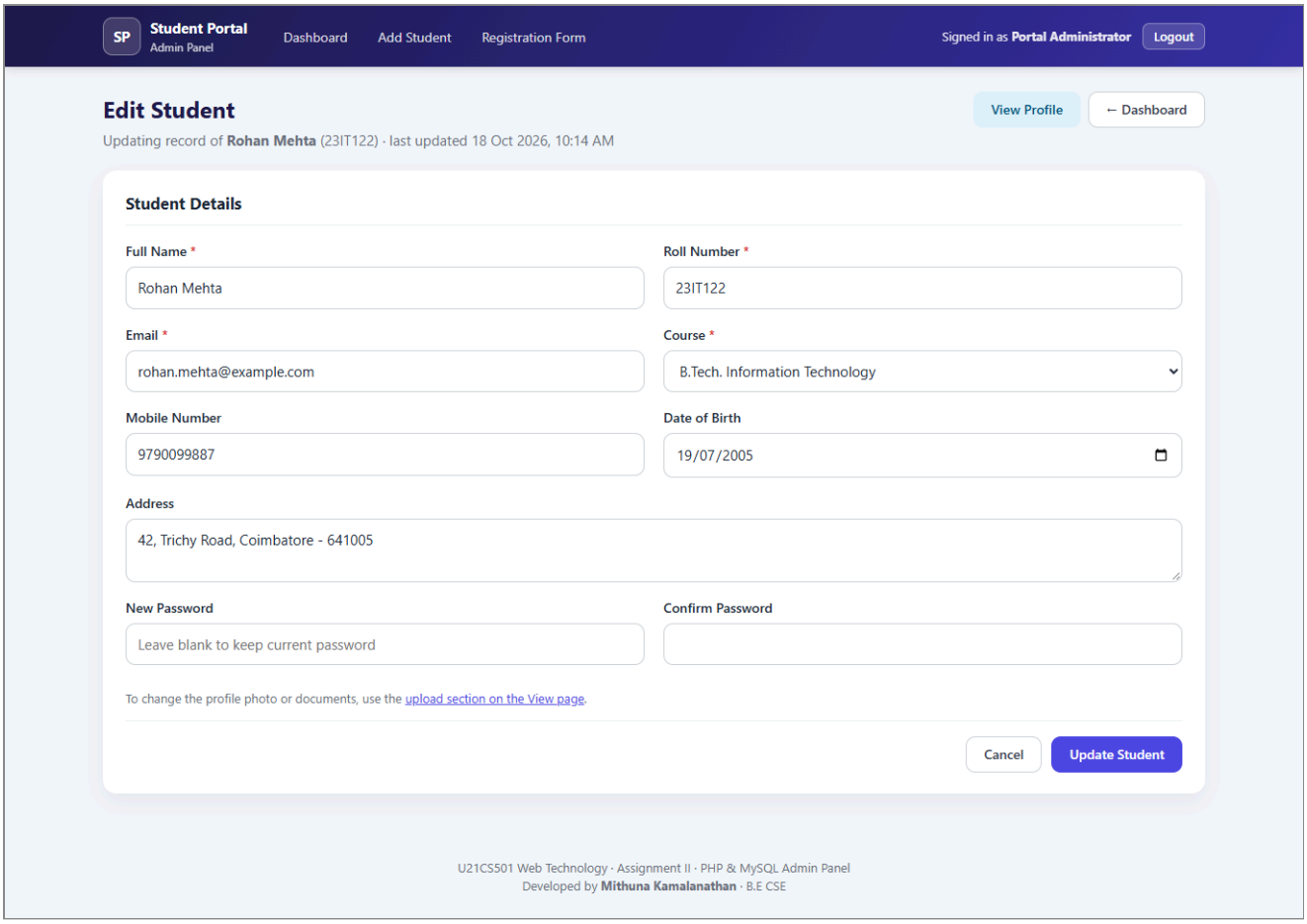


Figure 3.7 – Edit Student (Update)

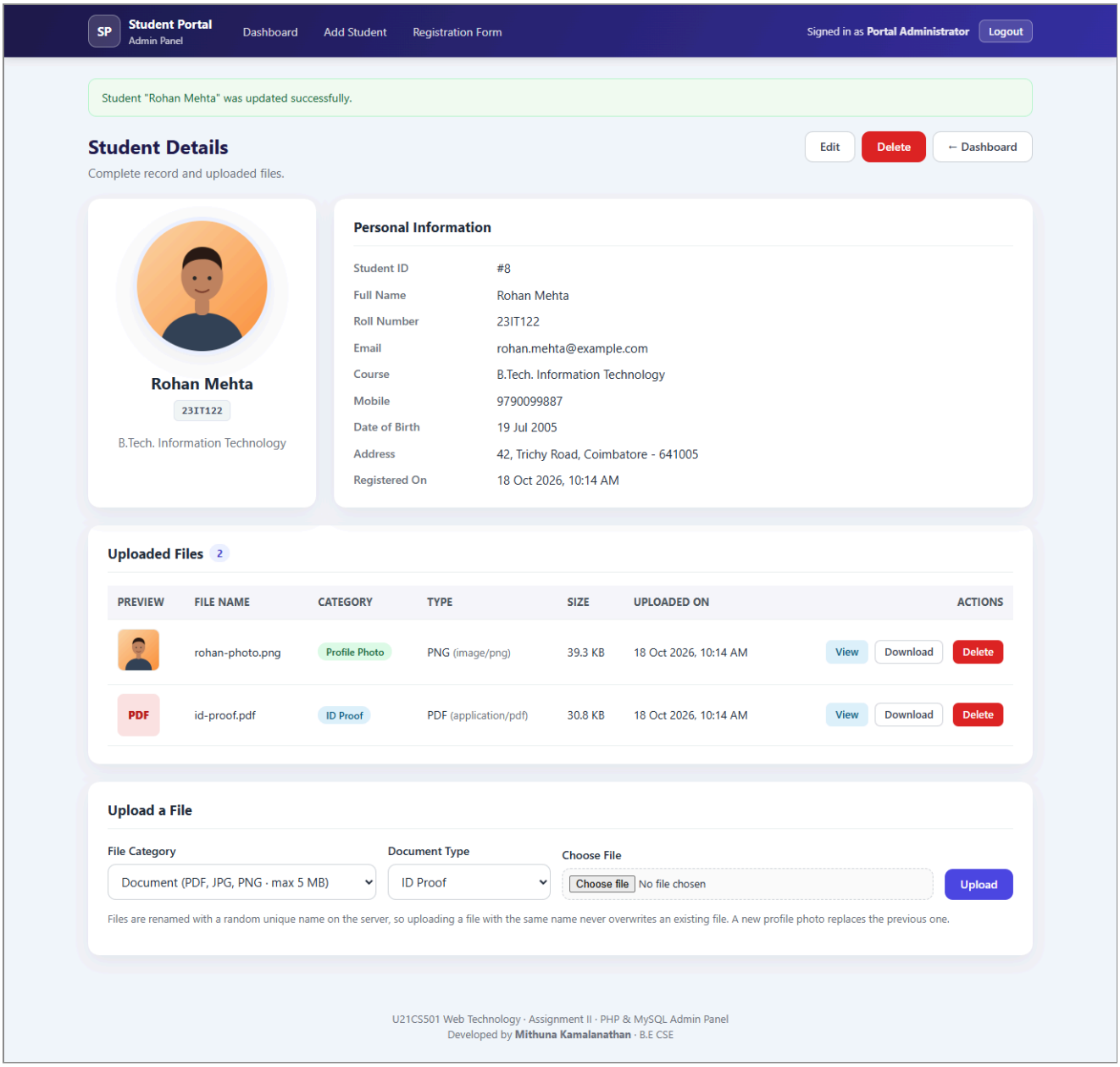


Figure 3.8 – View Student after update: details, profile image and uploaded files

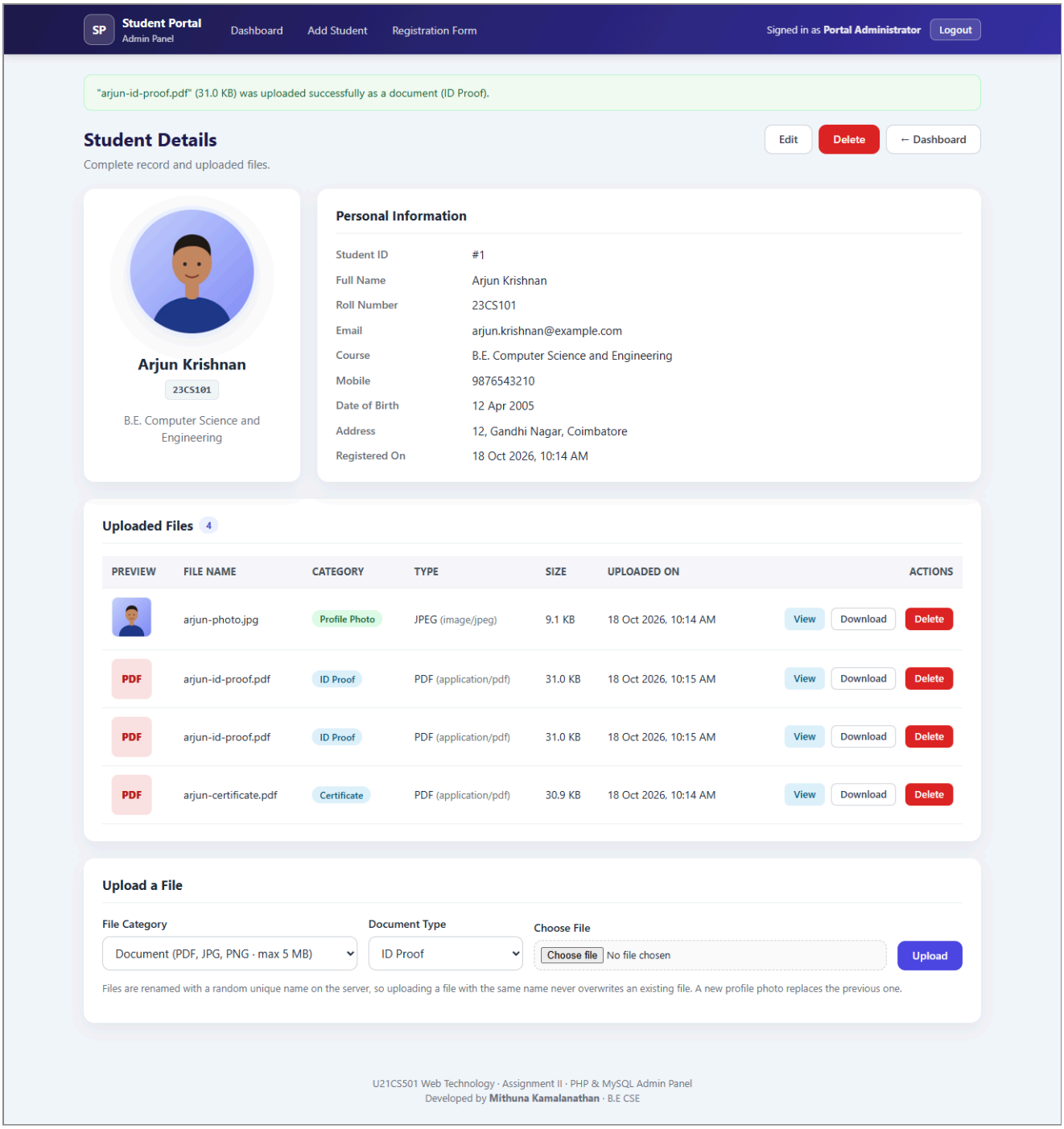


Figure 3.9 – Uploaded files with type, size and upload date; the same file name uploaded twice is stored twice (no overwriting)



Figure 3.10 – Unsupported file type (.txt) rejected



Figure 3.11 – File larger than 5 MB rejected



Figure 3.12 – Text file renamed to .jpg rejected by the MIME check



Figure 3.13 – `shell.php.jpg` rejected (dangerous extension)

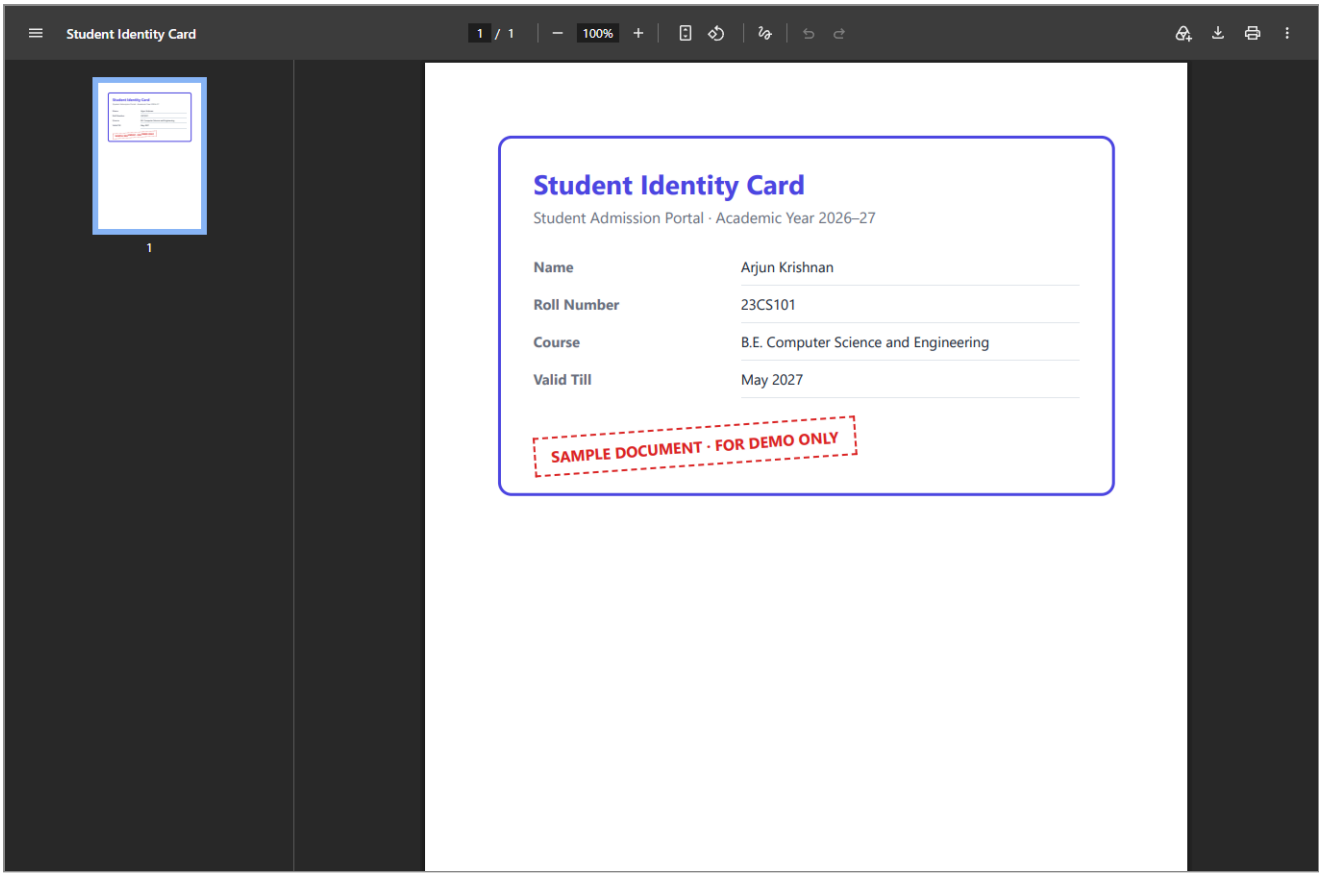


Figure 3.14 – Uploaded PDF displayed through `view_file.php`

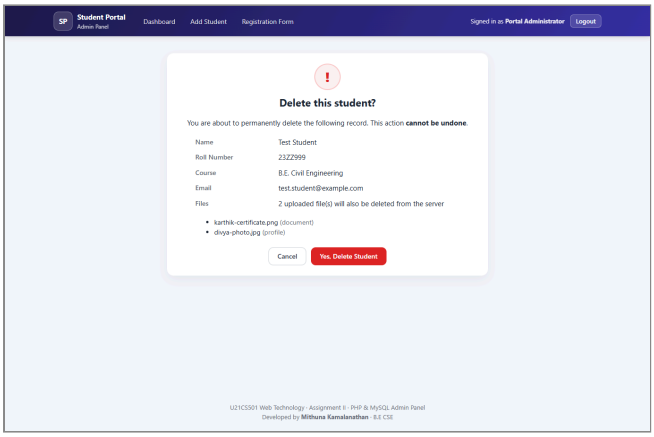


Figure 3.15 – Delete confirmation listing the files to be removed

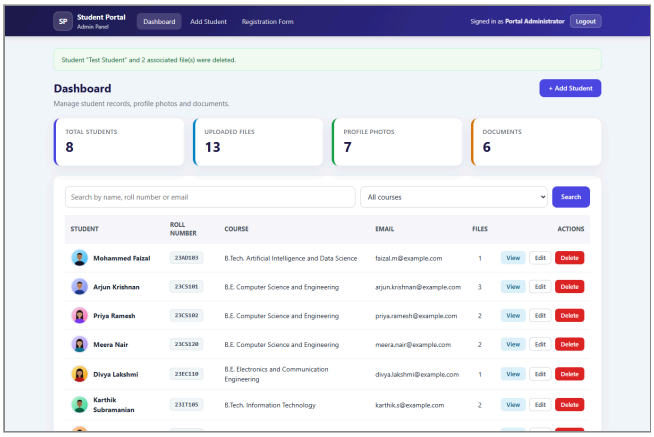


Figure 3.16 – Student and files deleted

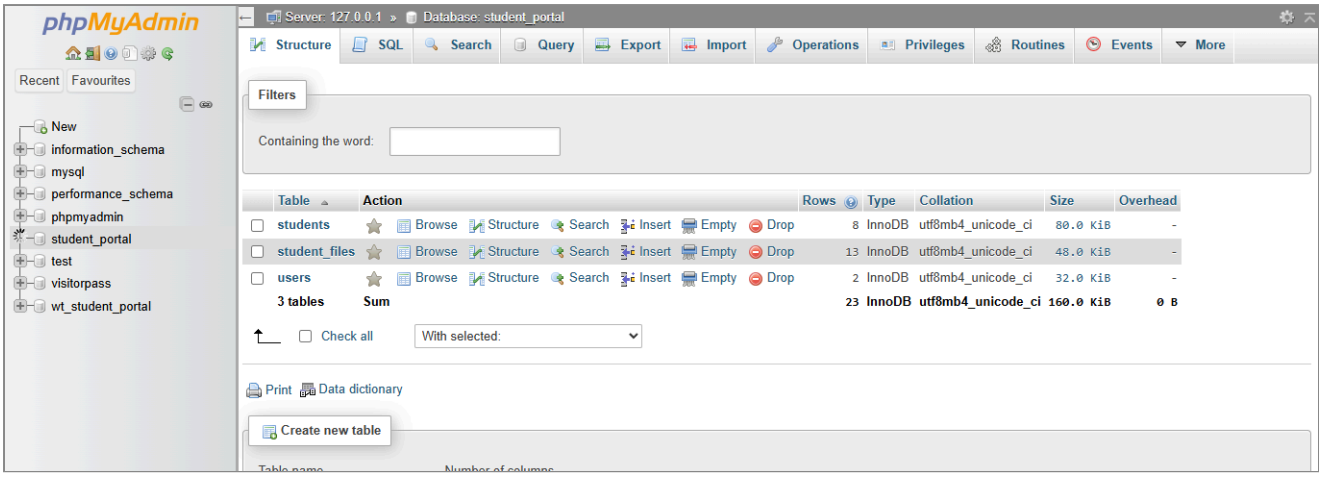


Figure 3.17 – MySQL tables of the `student_portal` database (phpMyAdmin)

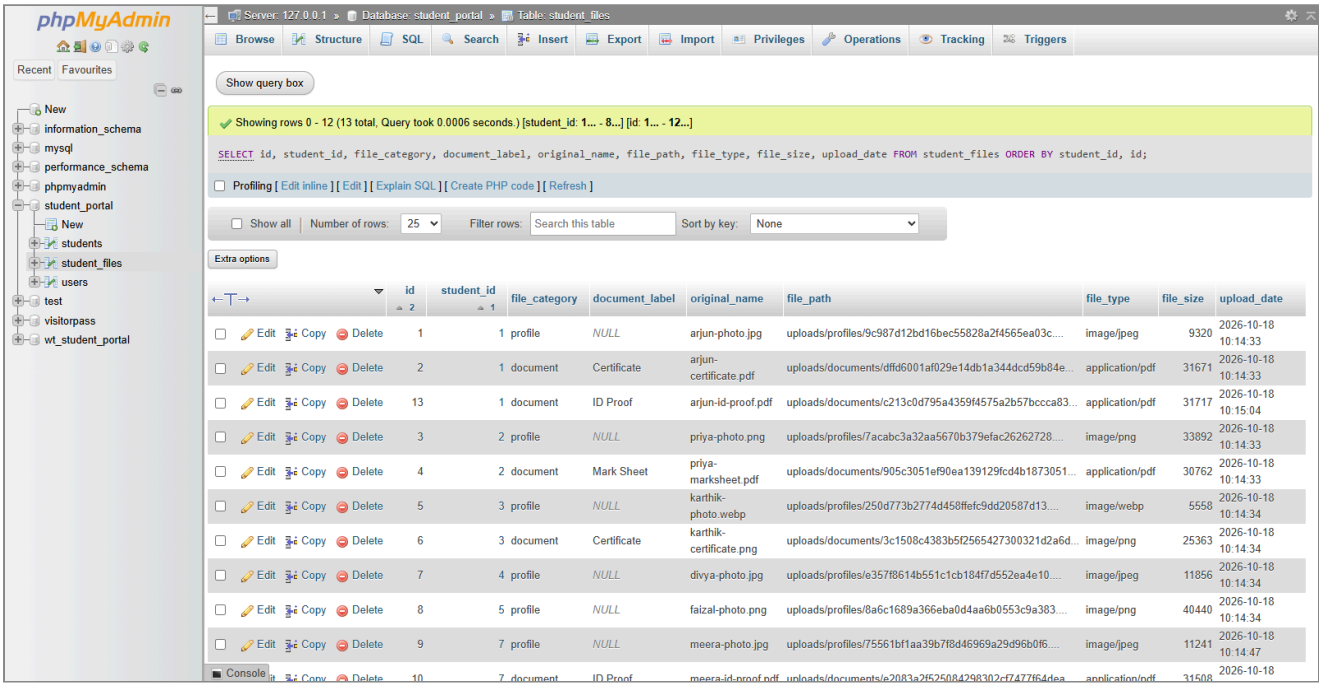


Figure 3.18 – File metadata stored in MySQL: student ID, file path, file type, size and upload date

Result

> The PHP admin panel performs Add, Edit, Delete and View operations on student records in MySQL using prepared statements. Profile images and documents are uploaded, validated by size, extension and real MIME type, stored under unique names so existing files are never overwritten, and their path, type, size and upload date are saved in MySQL and displayed on the student's page.

4. QUESTION 3 – Servlet, JSP & JDBC Portal (16 Marks)

Aim

To develop Servlet pages that connect to MySQL with JDBC to fetch student details and file paths, stream the stored images and documents, display student profiles dynamically in JSP with `<img>` tags pointing to the Servlet, use looping and conditional logic for multiple students and files, and allow only logged-in users through session handling.

Implementation (MVC structure)

Class / page	Role
`DBConnection`	Reads `db.properties`, loads the MySQL driver, returns JDBC connections
`StudentDAO`, `FileDAO`, `UserDAO`	All SQL (PreparedStatement) for students, file metadata and login
`Student`, `StudentFile`, `User`	Model classes (JavaBeans) used by the JSP pages
`LoginServlet` (`/login`)	Checks username/password (bcrypt) and creates the `HttpSession`
`LogoutServlet` (`/logout`)	Invalidates the session
`StudentListServlet` (`/students`)	Session check → loads all students → forwards to `students.jsp`
`StudentProfileServlet` (`/student?id=`)	Session check → loads one student and all files → `student-profile.jsp`
`FileDownloadServlet` (`/file?id=`)	Session check → finds the file safely → streams it with `ServletOutputStream`
`WEB-INF/web.xml`	Servlet mappings, session timeout, error pages; JSPs are kept in `WEB-INF/views`

Code Explanation

The excerpts below are copied from the project source and shortened; "..." marks omitted lines.

`Question3_Servlet/.../dao/FileDAO.java – JDBC query for a student's files`

```
String sql = "SELECT " + COLUMNS + " FROM student_files WHERE student_id = ? "
           + "ORDER BY (file_category = 'profile') DESC, upload_date DESC, id DESC";
try (Connection conn = DBConnection.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setInt(1, studentId);
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) { files.add(mapRow(rs)); }      // loop over every row
    }
}
```

`Question3_Servlet/.../util/SessionUtil.java – only logged-in users may continue`

```
HttpSession session = request.getSession(false);          // never creates a new session
if (session == null || session.getAttribute("loggedInUser") == null) {
    response.sendRedirect(request.getContextPath() + "/login?msg=login_required");
    return true;
}
```

Question3_Servlet/.../controller/FileDownloadServlet.java – safe path + streaming

```
Path baseDir    = Paths.get(DBConnection.getUploadBaseDir()).toRealPath();
Path uploadsDir = baseDir.resolve("uploads").toRealPath();
Path candidate  = baseDir.resolve(relativePath).normalize();
if (!candidate.startsWith(uploadsDir) || !Files.isRegularFile(candidate)) { return null; }  //
blocks ../ tricks
...
response.setContentType(contentType);                    // e.g. image/jpeg or application/pdf
response.setHeader("Content-Disposition", buildDisposition(disposition, file.getOriginalName()));
try (InputStream in = Files.newInputStream(filePath);
     ServletOutputStream out = response.getOutputStream()) {
    byte[] buffer = new byte[BUFFER_SIZE];
    int bytesRead;
    while ((bytesRead = in.read(buffer)) != -1) { out.write(buffer, 0, bytesRead); }
}
```

Question3_Servlet/.../WEB-INF/views/student-profile.jsp – looping and conditional rendering

```
<img class="profile-photo" src="${ctx}/file?id=${student.profilePhotoId}" alt="...">   <%-- points
to the Servlet --%>

<c:forEach items="${files}" var="f">
    <c:choose>
        <c:when test="${f.image}"> <img src="${ctx}/file?id=${f.id}" alt="..."> </c:when>
        <c:when test="${f.pdf}">   <span class="file-icon pdf">PDF</span>        </c:when>
        <c:otherwise>              <span class="file-icon">FILE</span>           </c:otherwise>
    </c:choose>
    <a href="${ctx}/file?id=${f.id}">View</a>  <a href="${ctx}/file?
id=${f.id}&amp;download=true">Download</a>
</c:forEach>
```

`students.jsp` uses `<c:forEach>` over all students and `<c:choose>` to show the photo (through `/file?id=…`) or the student's initials when no photo exists. All values are printed with `<c:out>` so they are HTML-escaped. On login the old session is invalidated and a new one is created (prevents session fixation), the session expires after 30 minutes, and protected pages are sent with `Cache-Control: no-store`.

Output

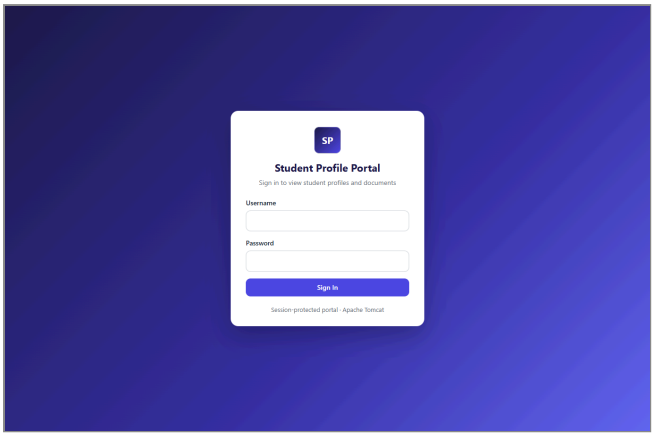


Figure 4.1 – Servlet login page

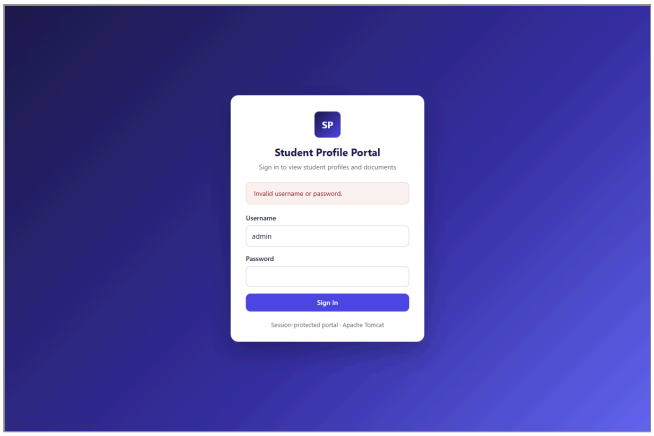


Figure 4.2 – Invalid login

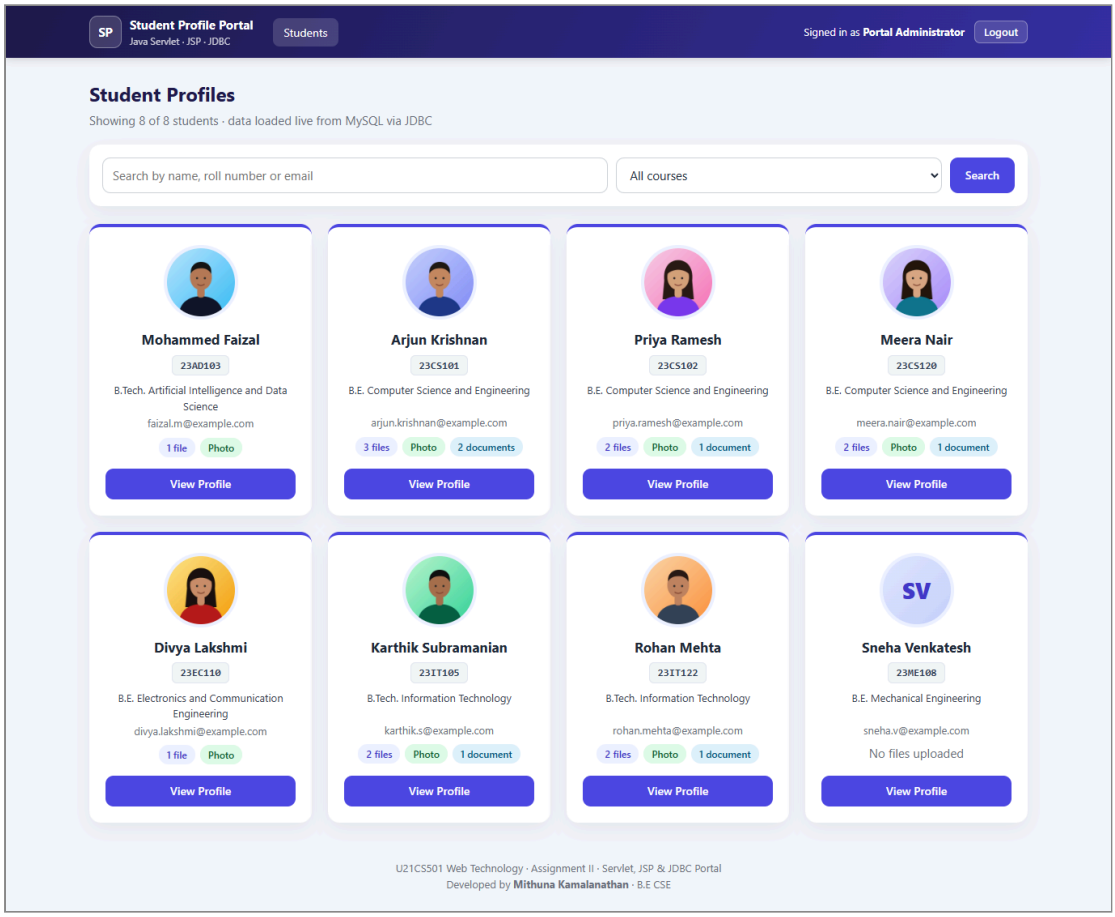


Figure 4.3 – Student list: loop over all students, photos streamed by the Servlet, initials when no photo

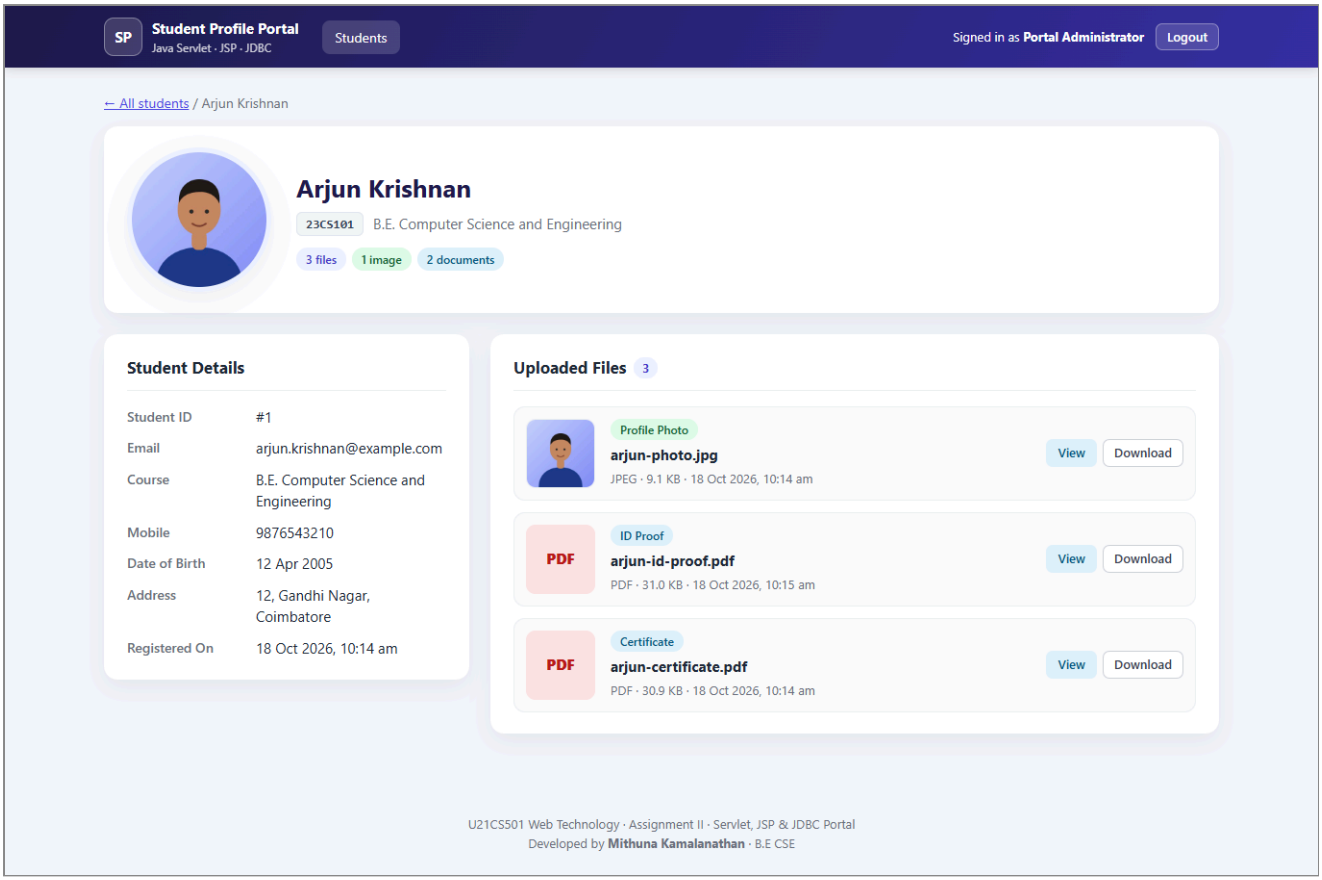


Figure 4.4 – Student profile: details and loop over multiple files (image preview / PDF icon)

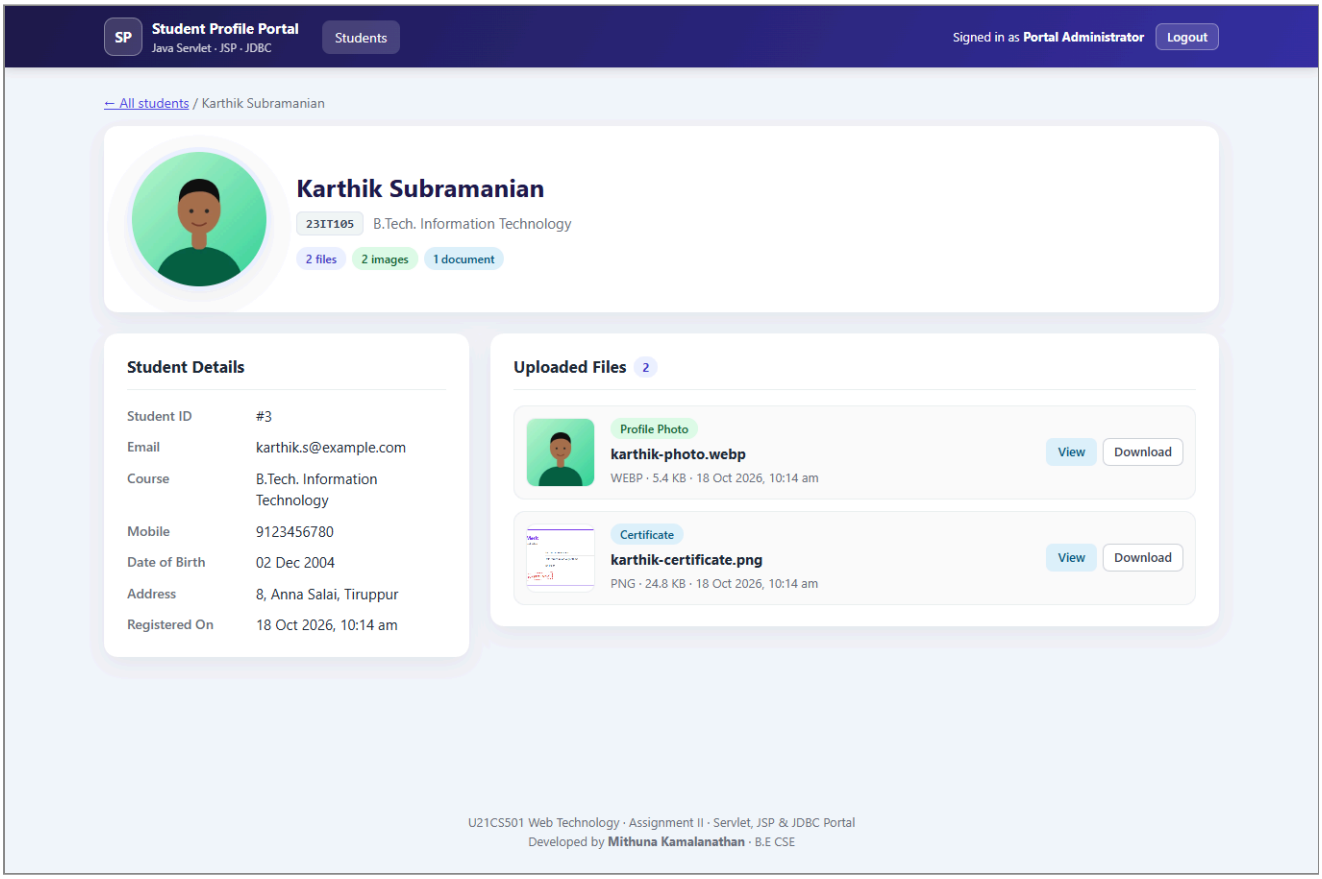


Figure 4.5 – Conditional rendering of a WEBP photo and a PNG certificate

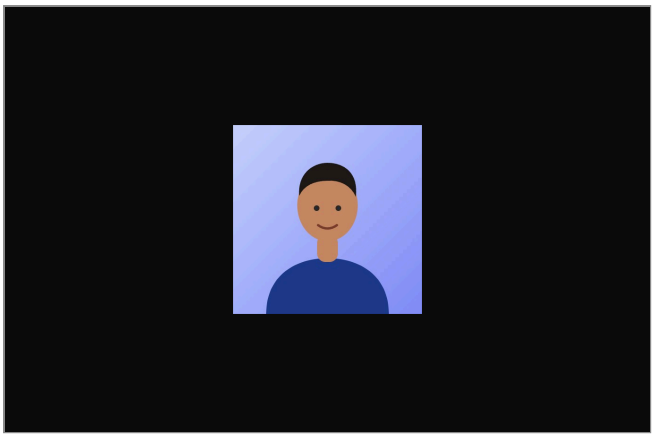


Figure 4.6 – Image streamed by `/file?id=1`

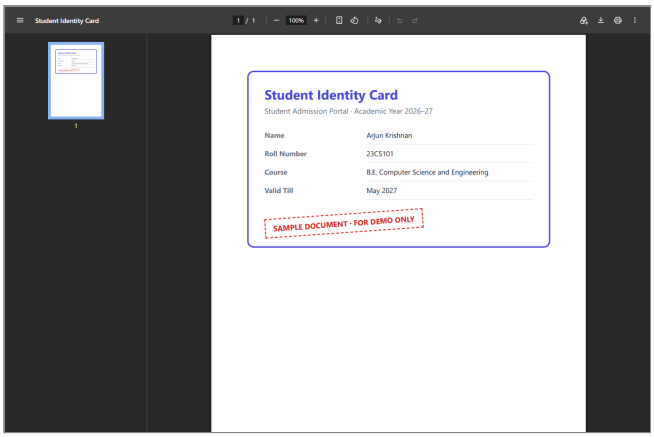


Figure 4.7 – PDF document streamed inline

```
Line wrap
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Arjun Krishnan | Student Profile Portal</title>
    <link rel="stylesheet" href="/student-portal/css/servlet.css">
</head>
<body>
<header class="topbar">
    <div class="container topbar-inner">
        <a class="brand" href="/student-portal/students">
            <span class="brand-logo" aria-hidden="true">SP</span>
            <span>
                <span class="brand-title">Student Profile Portal</span>
                <span class="brand-subtitle">Java Servlet · JSP · JDBC</span>
            </span>
        </a>
        <nav class="nav" aria-label="Portal navigation">
            <a href="/student-portal/students" class="active">Students</a>
        </nav>
        <div class="user-box">
                <span class="user-name">Signed in as
                    <strong>Portal Administrator</strong></span>
                <form method="post" action="/student-portal/logout" class="inline-form">
                    <button type="submit" class="btn btn-light btn-sm">Logout</button>
                </form>
            </div>
        </div>
</header>
<main class="container page">
<p class="breadcrumb"><a href="/student-portal/students">&larr; All students</a> / Arjun Krishnan</p>

<section class="profile-header card">
    <img class="profile-photo" src="/student-portal/file?id=1"
              alt="Profile photo of Arjun Krishnan">
        <div class="profile-summary">
        <h1>Arjun Krishnan</h1>
        <p><span class="badge">23CS101</span>
           <span class="muted">B.E. Computer Science and Engineering</span></p>
        <div class="chips">
            <span class="chip">3 files</span>
            <span class="chip chip-green">1 image</span>
            <span class="chip chip-blue">2 documents</span>
        </div>
    </div>
</section>

<div class="profile-layout">
    <section class="card">
        <h2 class="section-title">Student Details</h2>
        <dl class="detail-list">
```

Figure 4.8 – Generated HTML: `<img src="/student-portal/file?id=1">` points to the Servlet (no file path exposed)

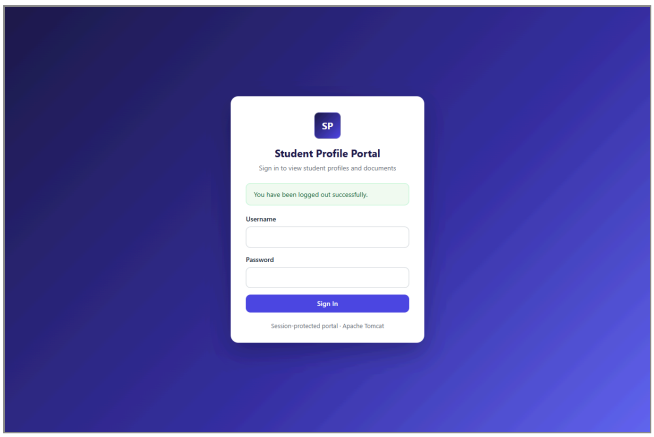


Figure 4.9 – Logout invalidates the session

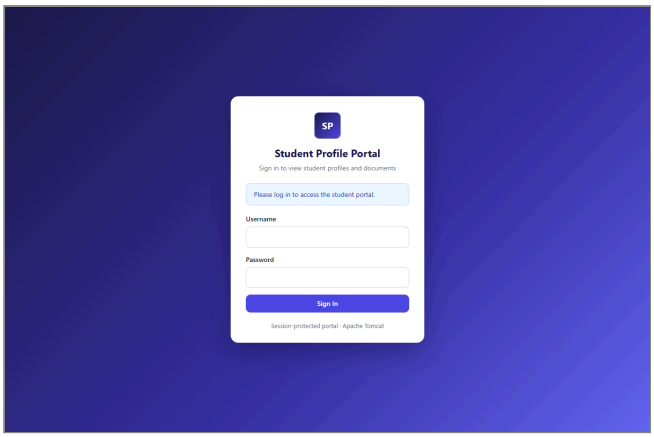


Figure 4.10 – Protected page without login redirects to the login page

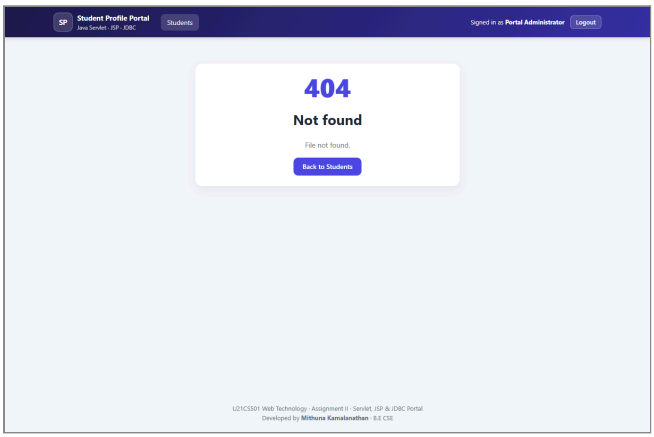


Figure 4.11 – Friendly 404 page for a missing file

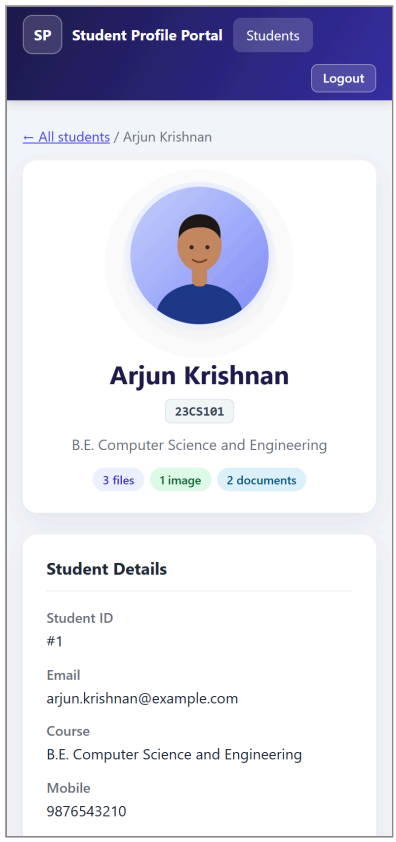


Figure 4.12 – Responsive JSP page on mobile

Result

> The Servlet portal connects to MySQL with JDBC, fetches students and their file paths, and displays them dynamically in JSP using JSTL loops and conditions. Images and documents are streamed from the upload folder by `FileDownloadServlet` and shown through `<img>` tags and links that point to the Servlet. Session handling allows only logged-in users to access the portal.

5. Database Schema

The script `database/schema.sql` creates the database `student_portal`, its three tables with primary keys, unique keys, a foreign key and check constraints, two demo login accounts and six sample students. Passwords are stored only as bcrypt hashes.

Table	Important columns	Constraints
`users`	id, username, full_name, password_hash, created_at, last_login	PK id; UNIQUE username
`students`	id, name, roll_number, email, course, phone, date_of_birth, address, password_hash, created_at, updated_at	PK id; UNIQUE roll_number; UNIQUE email
`student_files`	id, student_id, file_category, document_label, original_name, stored_name, file_path, file_type, file_size, upload_date	PK id; FK student_id → students.id ON DELETE CASCADE; UNIQUE stored_name; CHECK category and size

database/schema.sql – file metadata table

```
CREATE TABLE student_files (
    id              INT             NOT NULL AUTO_INCREMENT,
    student_id      INT             NOT NULL,
    file_category   VARCHAR(20)     NOT NULL,
    document_label  VARCHAR(50)     NULL,
    original_name   VARCHAR(255)    NOT NULL,
    stored_name     VARCHAR(100)    NOT NULL,
    file_path       VARCHAR(255)    NOT NULL,
    file_type       VARCHAR(100)    NOT NULL,
    file_size       BIGINT UNSIGNED NOT NULL,
    upload_date     TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    CONSTRAINT uq_files_stored_name UNIQUE (stored_name),
    CONSTRAINT fk_files_student FOREIGN KEY (student_id)
        REFERENCES students (id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_files_category CHECK (file_category IN ('profile', 'document')),
    CONSTRAINT chk_files_size CHECK (file_size > 0)
) ENGINE = InnoDB;
```

6. Security Measures

Threat	Protection used
SQL injection	Prepared statements in PHP (PDO) and Java (PreparedStatement)
Password theft	bcrypt hashes (`password_hash()` , `password_verify()` , jBCrypt); same error for wrong user or password
Cross-site scripting	`htmlspecialchars()` , `<c:out>` , `textContent`
Cross-site request forgery	Token in every PHP POST form, compared with `hash_equals()`
Malicious uploads	Size limits, extension whitelist, MIME check, random names, no-execute uploads folder
Path traversal	`realpath()` / `toRealPath()` and a check that the file stays inside `uploads/`
Unauthorised access	Login sessions in PHP and Java, JSPs inside `WEB-INF` , `.htaccess` deny rules
Information leakage	Friendly error pages; technical details only in the server logs

7. Testing Summary

71 test cases were executed on the running application (18 for the frontend, 29 for PHP and 24 for the Servlet portal); all of them passed. The complete table with input, expected result, actual result and status is in `docs/TESTING.md` . Important cases:

ID	Test	Actual result	Status
F02–F10	Invalid name, roll, email, phone, course, weak password, mismatch, file type	Correct message under each field; form not submitted	PASS
F13–F14	Valid registration	All fields green; saved by PHP	PASS
P04	Duplicate roll number / invalid email (server side)	Rejected with messages	PASS
P05–P09	Add, list, search, edit, view student	All operations successful	PASS
P10–P11	Upload image and PDF	Stored under random names; metadata saved	PASS
P12–P15	.txt file, 6 MB PDF, fake JPG, `shell.php.jpg`	All rejected	PASS
P16	Same file name uploaded twice	Both kept with different stored names	PASS
P20	Delete student with files	Record, metadata and files removed	PASS
P21–P28	Direct uploads access, no CSRF token, SQL injection, XSS, path traversal	Blocked (403 / rejected / escaped / 404)	PASS
S03–S04	Servlet login; protected pages without session	Session created; others redirected to login	PASS
S05–S08	Student list and profiles	Loops and conditions render correctly	PASS
S09–S13	Image and PDF streaming, forced download	Correct Content-Type and Content-Disposition	PASS
S21–S22	Logout, reuse of old session	Session invalidated; access denied	PASS

8. Conclusion

In this assignment I built a working student admission portal that combines a responsive HTML/CSS registration form with JavaScript validation, a PHP–MySQL admin panel for full CRUD operations and secure profile-image/document uploads, and a Java Servlet/JSP portal that reads the same database through JDBC and streams files through a Servlet. Uploaded files are validated by size, extension and real MIME type, stored under unique names so nothing is overwritten, and their metadata is linked to each student record. Prepared statements, password hashing, output escaping, CSRF tokens and session checks protect the system against common attacks and unauthorised access. Building it helped me understand how client-side scripting, server-side PHP, Java web components and a relational database work together in one application.

9. GitHub Repository and Submission Files

GitHub Repository: https://github.com/Mithu2706/WT_Assignment

The repository contains the complete source code, the database schema and all screenshots:

Submission requirement	Location in the repository
HTML, CSS and JavaScript files	`Question1_Frontend/`
PHP files	`Question2_PHP/`
Servlet and WEB-INF files	`Question3_Servlet/src/main/java`, `Question3_Servlet/src/main/webapp/WEB-INF/`
DB schema script	`database/schema.sql`
Screenshots of output	`Screenshots/` (40 images, described in `Screenshots/README.md`)
Code explanation (inline comments)	Comments in every source file and `docs/CODE_EXPLANATION.md`
Conclusion	Section 8 of this report and `docs/CONCLUSION.md`
Setup instructions and demo login	`README.md` (local demo: `admin` / `Admin@123`)